

A PROJECT REPORT ON

FINSIGHTS

Smart Finance Insights

Complete Project Report — Milestones 1 to 4

Submitted by

Biswajeet Behera

Under the guidance of

Dr. V. Suresh Kumar

Abstract

FinSights (developed under the working title 'Smart Finance Insights' during early prototyping) is a Python desktop application that brings together the everyday pieces of personal finance — income, expenses, budgets, savings goals and investments — into a single, self-contained tool. It is built with CustomTkinter for the interface and stores all data locally as JSON files, so it works fully offline and does not ask the user to hand over any bank credentials.

The project was carried out in four milestones under the guidance of Dr. V. Suresh Kumar. Milestone 1 laid the foundation — account registration, login, the dashboard shell, and basic expense/income logging. Milestone 2 extended this into budgeting, goal planning, investment tracking, a computed financial health score, exportable reports, notifications, an admin panel and a profile section. Milestone 3 added an intelligence layer on top of that data — a dedicated analytics dashboard, AI-style spending insights, and a refined, more detailed health score. Milestone 4 rounded the application off with predictive forecasting, a net-worth tracker, recurring-transaction automation, data backup/restore, an application settings panel, and general polish across the reporting and profile screens, including a light theme option.

This report walks through the reasoning, design and implementation behind each milestone, backed by screenshots taken directly from the working application, and closes with the challenges faced along the way and the scope for future work.

Table of Contents

- Abstract** 2
- 1. Introduction** 5
 - 1.1 Background..... 5
 - 1.2 Problem Statement 5
 - 1.3 Objectives 5
 - 1.4 Scope of this Report 5
- 2. Tools and Technologies Used**..... 6
- 3. System Design** 6
 - 3.1 Architectural Approach 6
 - 3.2 Data Storage 6
 - 3.3 Module Overview 6
 - 3.4 Database Schema 8
- 4. Milestone 1: Core Foundation** 10
 - 4.1 User Registration and Login 10
 - 4.2 Dashboard Shell..... 11
 - 4.3 Expense Tracking 12
 - 4.4 Income Tracking 12
- 5. Milestone 2: Extended Features** 14
 - 5.1 Budget Management..... 14
 - 5.2 Goal Planning..... 15
 - 5.3 Investment Portfolio 16
 - 5.4 Financial Health Score 18
 - 5.5 Reports 19
 - 5.6 Notifications, Admin Panel and Profile 19
- 6. Milestone 3: Intelligence & Insights** 20
 - 6.1 Analytics Dashboard..... 20
 - 6.2 AI-Powered Insights..... 20
 - 6.3 Refined Financial Health Score..... 21
- 7. Milestone 4: Reporting, Automation & Finalization** 23
 - 7.1 Advanced Reports 23
 - 7.2 Financial Forecast..... 23
 - 7.3 Net Worth Tracker..... 24
 - 7.4 Recurring Transactions 24
 - 7.5 Backup & Restore 25

7.6 Settings, Profile and Theming	26
8. Challenges Faced	28
9. Conclusion and Future Scope	28
References	28

1. Introduction

1.1 Background

Every semester I've noticed the same thing happen to my own wallet — expenses pile up across a dozen small transactions and by the time I sit down to actually check where the money went, half of it is already a mystery. Most students and young professionals I know track money either in a scattered set of notes-app entries or not at all, and the handful of finance apps available either push you toward a bank integration you don't trust or bury the useful analytics behind a subscription. That gap is what pushed me toward this project: a personal finance manager that runs entirely on the desktop, keeps all data local, and still gives a proper picture of income, spending, goals and investments in one place.

FinSights (branded internally during early prototyping as 'Smart Finance Insights') is the result of that idea — a Python desktop application built with CustomTkinter for the interface. I picked a desktop-first approach deliberately rather than jumping straight to a web app, mainly because it let me focus on getting the core logic and UI right without also having to handle hosting, authentication over a network, or browser quirks.

1.2 Problem Statement

Individuals managing personal finances across income, expenses, budgets, savings goals and investments typically rely on either fragmented tools (a spreadsheet for expenses, a separate app for investments, a mental note for goals) or on finance apps that demand sensitive bank credentials. There is a need for a single, self-contained application that consolidates these categories, gives the user visual feedback on their financial health, and does not require any external account or internet dependency to function.

1.3 Objectives

- Build a desktop application that lets a user record and categorise income and expense transactions.
- Provide budget creation and tracking per category, with visual indicators of overspending.
- Allow users to define and monitor progress toward savings goals.
- Track investment holdings across different instrument types.
- Compute and display an overall financial health score derived from the user's data.
- Generate exportable reports (PDF/Excel) summarising the user's financial activity.
- Keep the codebase modular so that new features can be added without reworking the whole dashboard.

1.4 Scope of this Report

This report documents the work completed across all four milestones of the project, as guided by Dr. V. Suresh Kumar. Milestone 1 covers the foundational pieces — user registration, authentication, the dashboard shell, and expense/income tracking. Milestone 2 builds on top of that foundation to add budgeting, goal planning, investment tracking, the financial health score, reporting, notifications, an admin panel, and the user profile section. Milestone 3 adds an intelligence layer — a dedicated

analytics dashboard, AI-style spending insights, and a more detailed health score breakdown. Milestone 4 closes out the project with forecasting, net worth tracking, recurring-transaction automation, backup/restore, an application settings panel, and general refinement of reporting and profile screens.

2. Tools and Technologies Used

I kept the tech stack intentionally lightweight so I could iterate quickly without fighting a heavy framework. Everything runs locally with no server component.

Language	Python 3
GUI Framework	CustomTkinter (built on Tkinter)
Supplementary UI	Standard Tkinter ttk widgets (Treeview, styles) for tables
Data Storage	Local JSON files (transactions, budgets, goals, investments, notifications, users)
Charts / Visualisation	Matplotlib, embedded inside the dashboard canvases
Report Export	ReportLab (PDF export), OpenPyXL (Excel export)
Date Handling	python-dateutil (recurring transaction scheduling)
Version Control	Git / GitHub

3. System Design

3.1 Architectural Approach

The codebase follows a loosely MVC-inspired structure. The dashboard window (`dashboard.py` and `base_dashboard.py`) acts as the shell — it owns the window, sidebar navigation, theming, and shared helper methods. Each functional area of the app (expenses, income, budget, goals, investments, health, notifications, reports, admin, profile) lives in its own file under the `modules/` folder and is written as a mixin class that gets combined into the final `FinsightsDashboard` class. This kept individual files small and let me work on one feature at a time — say, budgeting — without touching the code for goals or investments at all, which mattered a lot once the project crossed a few thousand lines.

`config.py` is the closest thing to a 'model' layer: it centralises the colour palette, sizing constants, category lists, currency formatting helpers, and — importantly — the load/save functions (`_ld`, `_sv`, `_ldd`) that every module uses to read from and write to the JSON data files. Having one place responsible for persistence meant that whenever I changed how a record was stored, I only had to touch one file instead of hunting through every module.

3.2 Data Storage

Rather than setting up a full database engine for what is, at this stage, a single-user desktop app, I stored data as flat JSON files under a `data/` directory — one file each for transactions, budgets, goals, investments, and notifications, plus a separate users file for registered accounts. Each record carries an id generated at creation time, which is what the edit/delete flows key off. This was simple to implement and easy to inspect/debug during development, at the cost of not scaling well to concurrent access — which is an acceptable trade-off for a personal, single-user tool.

3.3 Module Overview

<i>regform.py</i>	User registration and login screen, password hashing/verification
<i>dashboard.py / base_dashboard.py</i>	Main window shell, sidebar navigation, shared UI setup
<i>modules/overview.py</i>	Landing dashboard view — summary cards, quick stats
<i>modules/expenses.py</i>	Add / edit / delete expense entries
<i>modules/income.py</i>	Add / edit / delete income entries
<i>modules/budget.py</i>	Category-wise budget creation and progress bars
<i>modules/goals.py</i>	Savings goal creation and progress tracking
<i>modules/investments.py</i>	Investment holdings — add / edit / delete
<i>modules/health.py</i>	Financial health score computation and display
<i>modules/reports.py</i>	PDF / Excel report generation, chart rendering
<i>modules/notifications.py</i>	In-app notifications and alerts
<i>modules/admin.py</i>	Admin-only panel — user oversight, broadcast messages
<i>modules/profile.py</i>	User profile, linked accounts, password change, logout
<i>modules/analytics.py</i>	Full analytics dashboard — income/expense/savings charts and summary table
<i>modules/insights.py</i>	AI-style insights — pattern-based observations on spending behaviour
<i>modules/forecast.py</i>	Predictive income/expense forecast from historical monthly data
<i>modules/net_worth.py</i>	Net worth tracking — assets, liabilities and trend chart
<i>modules/recurring.py</i>	Recurring transaction rules and automatic generation
<i>modules/backup.py</i>	Export / import a full data backup as a ZIP archive
<i>modules/settings.py</i>	Application preferences — theme, default currency, alerts
<i>modules/transactions.py</i>	Unified, filterable view across income and expense records

3.4 Database Schema

Even though the current build persists data as flat JSON files rather than a live SQL database, I designed the underlying data model the way I would design a relational schema from the start — with a proper primary/foreign key structure, typed columns, and clearly separated tables per feature. Laying it out this way early on made the JSON structure predictable (each 'table' below maps almost one-to-one to a JSON file, or a JSON array per user), and it also means migrating to an actual SQLite or Postgres database later, if the app ever needs to support multiple concurrent users, is mostly a matter of standing up the tables as-is rather than redesigning the data model from scratch.

Every table in the schema hangs off the central USERS table through a user_id foreign key, which is what keeps one person's transactions, budgets, goals, investments and settings completely isolated from anyone else's — a Recurring_Transactions row, a Net_Worth record, a Backup_History entry all trace back to exactly one user. The relationships are mostly one-to-many: a single user can have many transactions, many budgets, many goals, many investments, many notifications and many backups, while Settings is deliberately one-to-one per user since there's only ever one active preference set at a time.

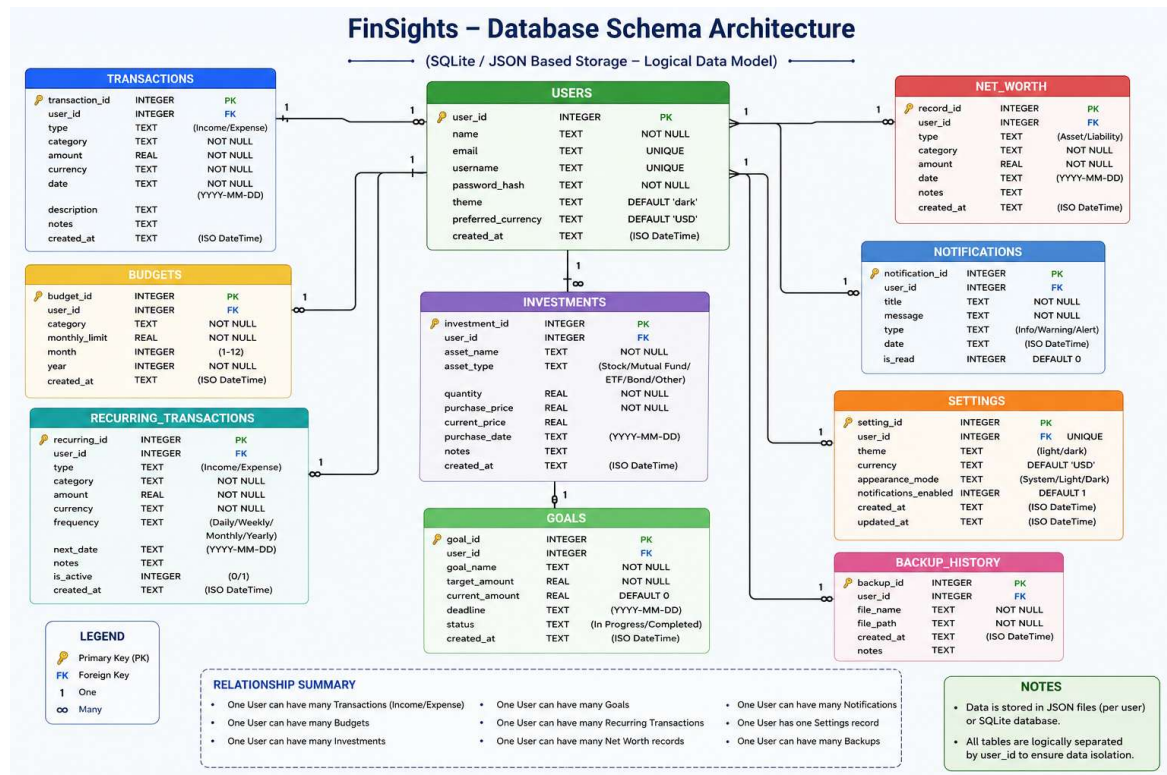


Fig 3.1 — FinSights database schema architecture (logical data model)

A few design choices worth calling out: amounts are stored as REAL rather than INTEGER since the app deals in currency values that can carry decimals, and every table carries its own created_at timestamp so records can be sorted and audited chronologically without relying on insertion order. The Transactions table doubles for both income and expense entries, distinguished by the type column, which kept the schema from needing two near-identical tables. Budgets are scoped to a specific month and year rather than being open-ended, which matches how the budgeting module actually works — a fresh budget per category, per month. The Settings table stores a redundant copy of theme and

currency alongside the same fields already present on Users; this was a deliberate, slightly imperfect trade-off so the settings screen could read and write a single focused record without touching the core user profile row.

4. Milestone 1: Core Foundation

Milestone 1 was about getting the skeleton of the app right — an account system, a working dashboard window, and the ability to log day-to-day income and expenses. Nothing here is flashy, but it's the part everything else in Milestone 2 depends on, so I spent more time than I expected just making sure the window handling, styling, and data flow were solid before adding more modules on top.

4.1 User Registration and Login

New users sign up through a registration form that collects the basic account details and stores credentials in the local users file. Passwords are never stored in plain text — I used HMAC-based hashing (`hash_password` / `verify_password` in `config.py`) so that even the local JSON file doesn't expose a readable password. Once registered, the same screen doubles as the login form for returning users.

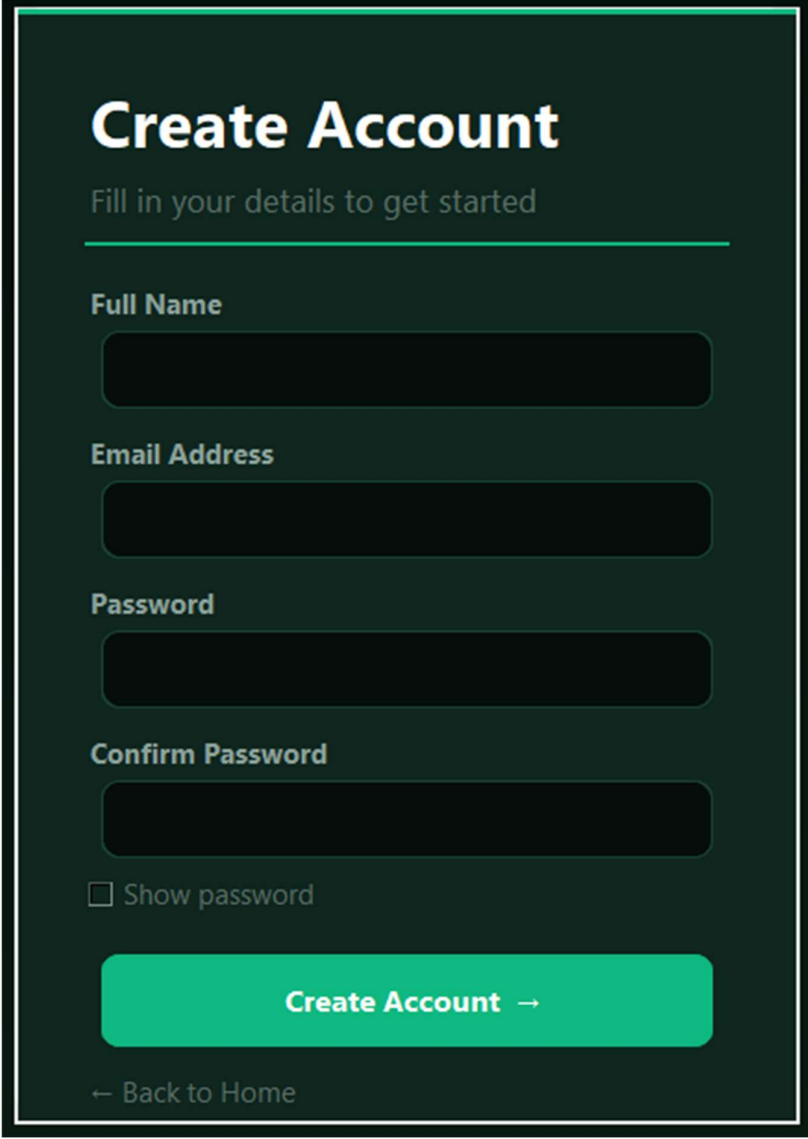
The image shows a user registration screen with a dark teal background. At the top, the title "Create Account" is in large white font, followed by the subtitle "Fill in your details to get started" in a smaller white font. Below this is a horizontal line. The form consists of four input fields, each with a label above it: "Full Name", "Email Address", "Password", and "Confirm Password". Each label is in white font. The input fields are dark teal with rounded corners. Below the "Confirm Password" field is a checkbox labeled "Show password" in white font. At the bottom of the form is a large orange button with the text "Create Account" and a right-pointing arrow. Below the button is a link that says "← Back to Home" in white font.

Fig 4.1 — User registration screen

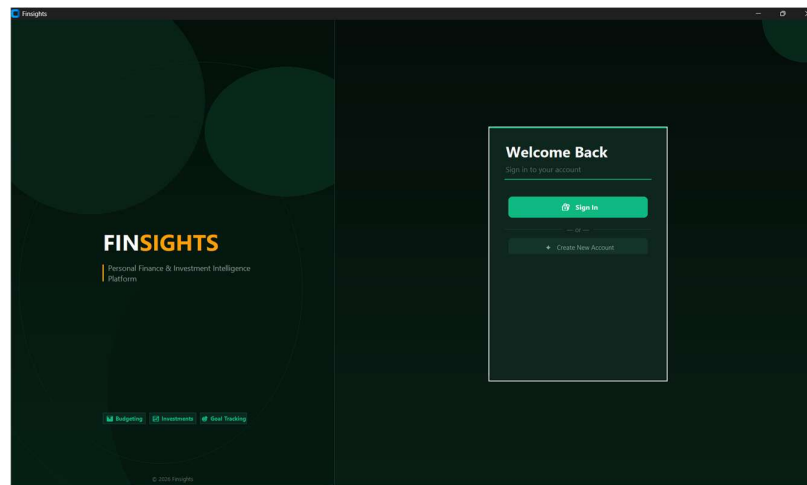


Fig 4.2 — Login screen

4.2 Dashboard Shell

After login, the user lands on the main dashboard window. This is where a lot of the early effort went — the sidebar navigation, the dark fintech-style theme, consistent card styling, scroll handling across different sections, and window resizing all had to be sorted out here before any individual module could be plugged in cleanly. The overview screen itself pulls together summary cards (income, expenses, balance, budget status) so the user gets a snapshot the moment they log in, rather than having to click into each section separately.

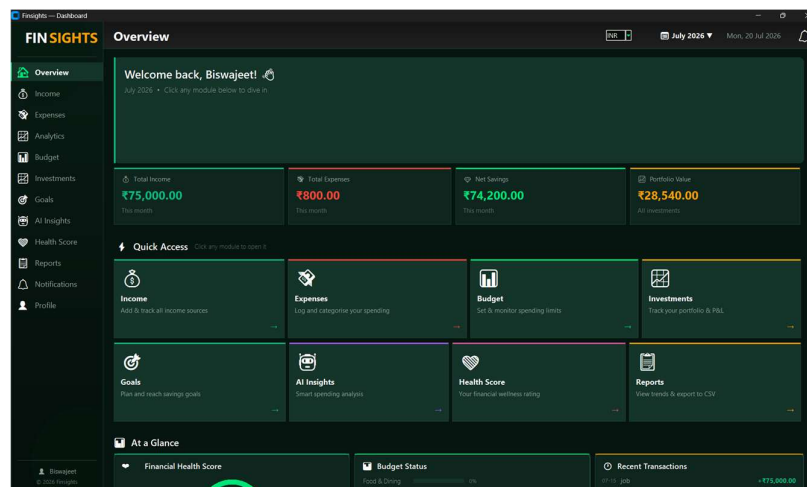


Fig 4.3 — Dashboard overview screen

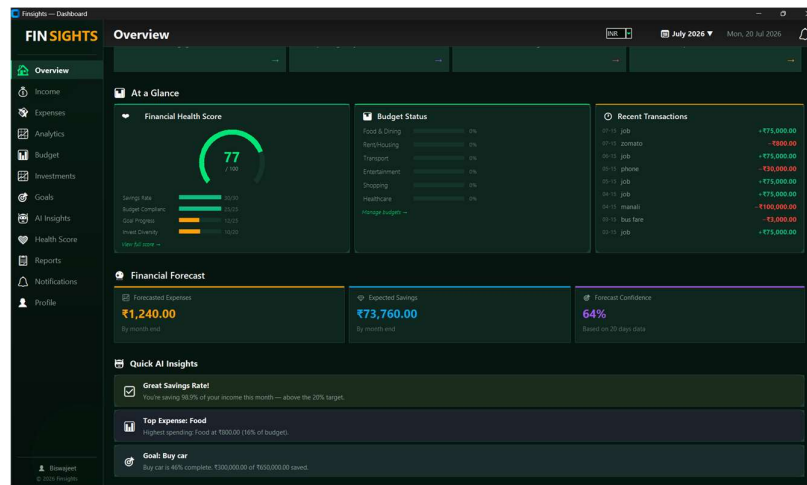


Fig 4.4 — Dashboard overview, alternate view

4.3 Expense Tracking

The expense module lets the user add a transaction with an amount, category, date and optional note, and lists all recorded expenses in a sortable table. Editing and deleting are handled inline rather than through a separate page, which keeps the interaction quick for something users will do often.

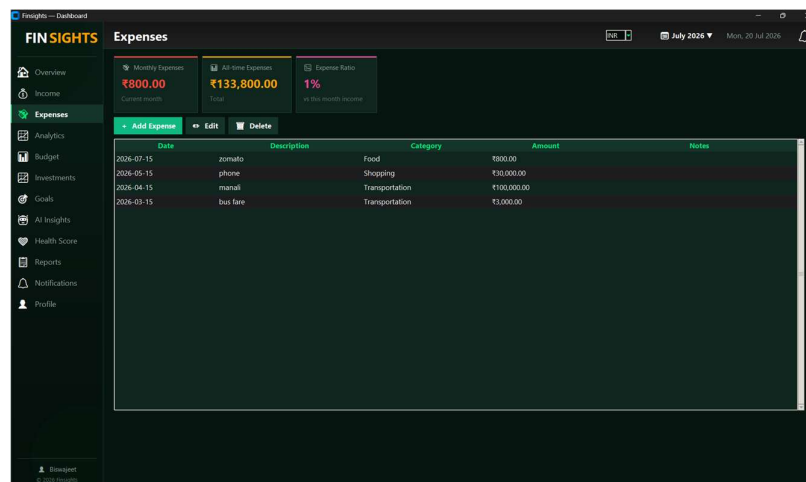


Fig 4.5 — Expense management screen

4.4 Income Tracking

Income entries follow the same pattern as expenses — add, edit, delete — but are kept in a separate category set (salary, freelance, investment returns, etc.) since income sources tend to be fewer and more predictable than expense categories. Keeping income and expenses as parallel-but-separate modules made the balance calculation on the overview page a straightforward subtraction rather than something that needed a shared transaction-type flag threaded through every screen.

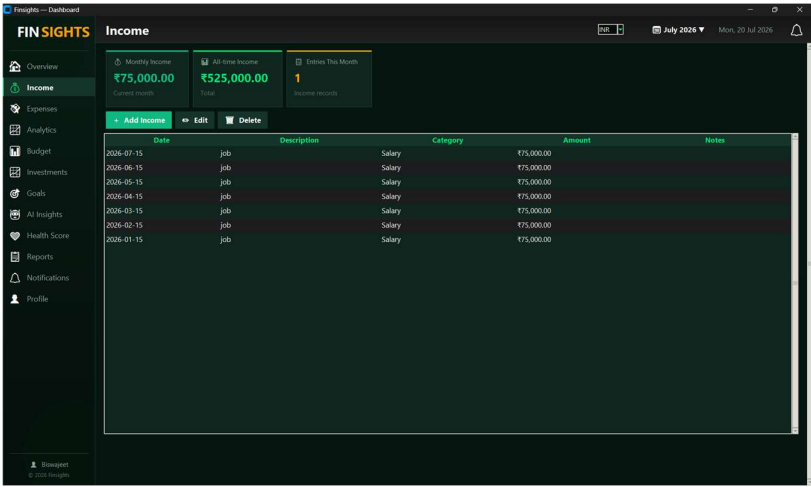


Fig 4.6 — Income management screen

5. Milestone 2: Extended Features

With the core in place, Milestone 2 focused on the features that make the app genuinely useful for planning rather than just record-keeping — budgets, goals, investments, a health score, reporting, and the supporting admin/notification/profile layers.

5.1 Budget Management

Users can set a spending limit per category for the current period. The budget screen compares actual spending (pulled from the expense records) against the set limit and renders a progress bar per category, colour-coded so it's obvious at a glance which categories are running over. Adding or editing a budget opens a small dialog rather than a full page, which matches the pattern used elsewhere in the app for quick data entry.

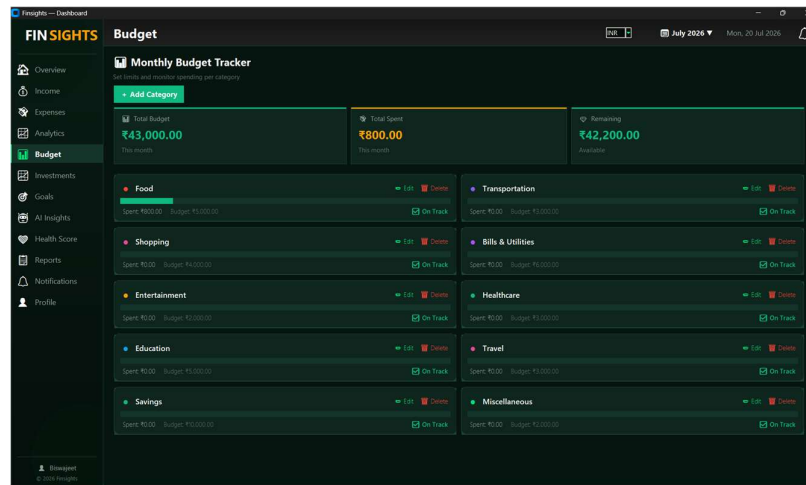
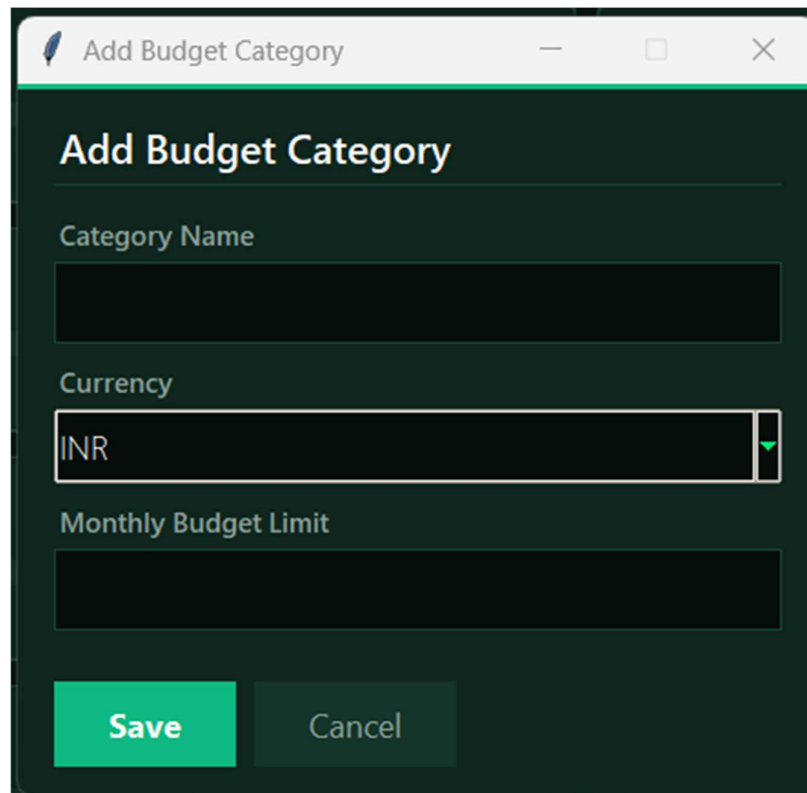


Fig 5.1 — Budget management screen



The image shows a dialog box titled "Add Budget Category". It has a dark green background with white text. The title is at the top in a large font. Below the title, there are three input fields: "Category Name", "Currency", and "Monthly Budget Limit". The "Currency" field is currently set to "INR" and has a dropdown arrow on the right. At the bottom of the dialog, there are two buttons: a green "Save" button and a grey "Cancel" button.

Fig 5.2 — Add / edit budget dialog

5.2 Goal Planning

The goals module lets a user define a savings target — say, an emergency fund or a purchase they're saving toward — along with a target amount and, optionally, a deadline. Progress is tracked against contributions and shown visually, which was one of the more satisfying pieces to build since it's the closest thing in the app to positive reinforcement for the user rather than just a ledger.

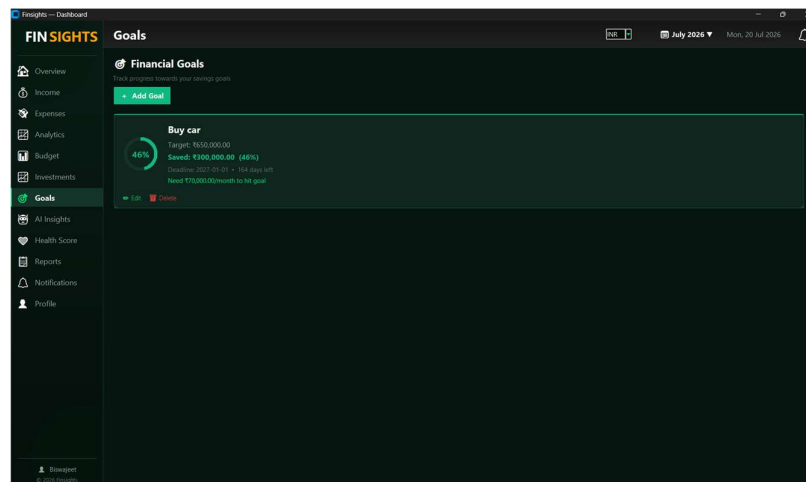
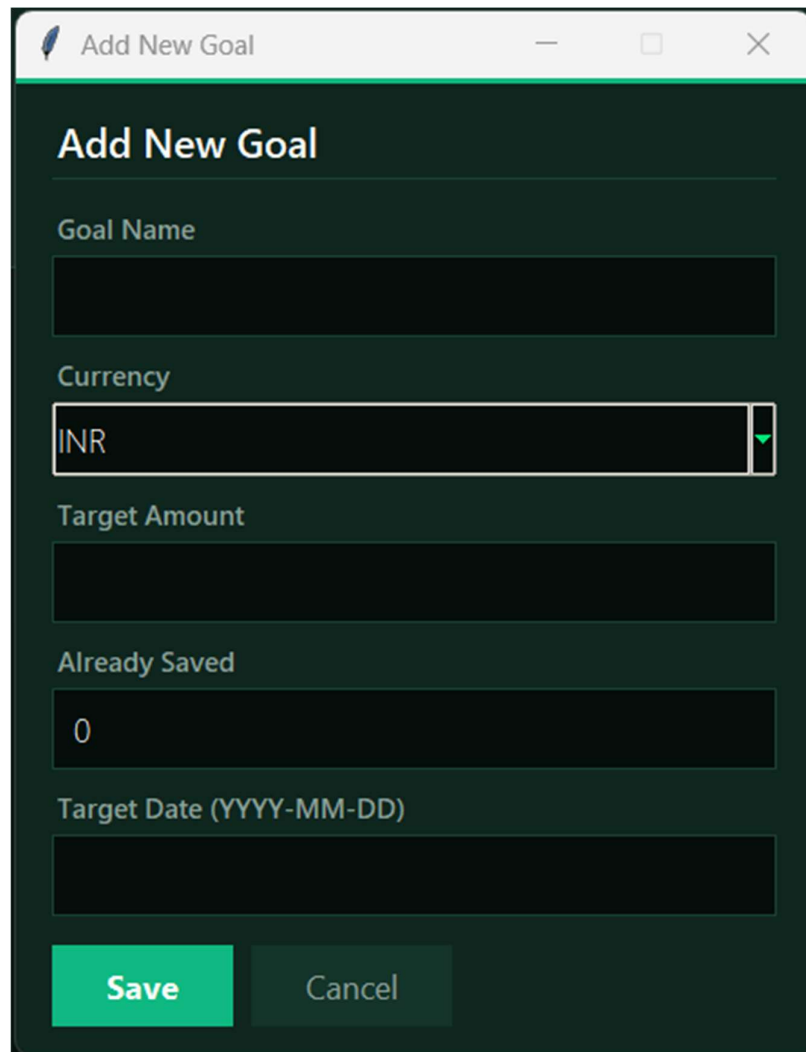


Fig 5.3 — Goal planning screen



The image shows a 'Add New Goal' dialog box with a dark green background. At the top, there is a title bar with a feather icon, the text 'Add New Goal', and standard window controls (minimize, maximize, close). The main content area has the title 'Add New Goal' in white. Below the title are five input fields: 'Goal Name' (a text box), 'Currency' (a dropdown menu showing 'INR' with a green arrow), 'Target Amount' (a text box), 'Already Saved' (a text box showing '0'), and 'Target Date (YYYY-MM-DD)' (a text box). At the bottom, there are two buttons: a green 'Save' button and a grey 'Cancel' button.

Fig 5.4 — Add / edit goal dialog

5.3 Investment Portfolio

This section tracks investment holdings — type of instrument, amount invested, and related details — so a user gets a consolidated view of where their money is deployed outside of day-to-day spending. It doesn't pull live market data (that would need a network dependency I deliberately avoided at this stage); instead, the user enters and updates values manually, which is a reasonable trade-off for a first version.

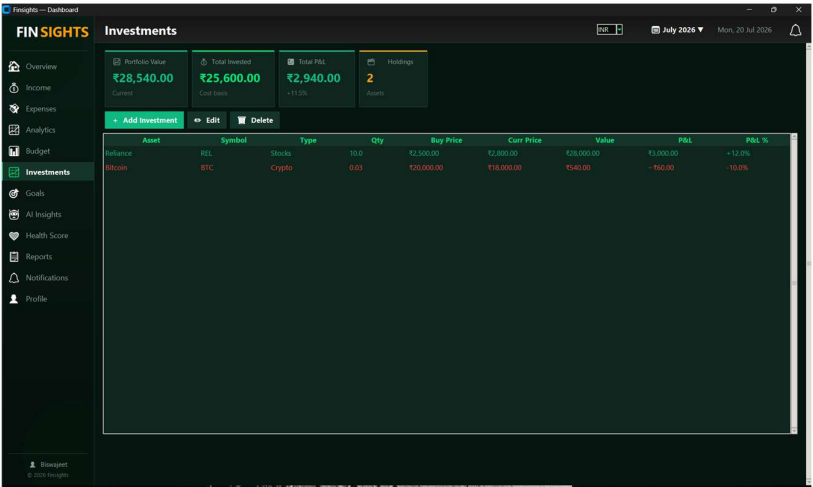


Fig 5.5 – Investment portfolio screen

Add Investment

Asset Name

Symbol / Ticker

Type

Stocks

Currency

EUR

Quantity

Buy Price

Current Price

Notes

Fig 5.6 — Add / edit investment dialog

5.4 Financial Health Score

One of the features I was most interested in building was a single score that reflects the user's overall financial standing, rather than making them interpret four different screens on their own. The health module pulls together signals from income vs. expenses, budget adherence, and goal progress into one score, displayed with a breakdown so the user can see which factor is dragging the number down instead of just being handed an opaque number.

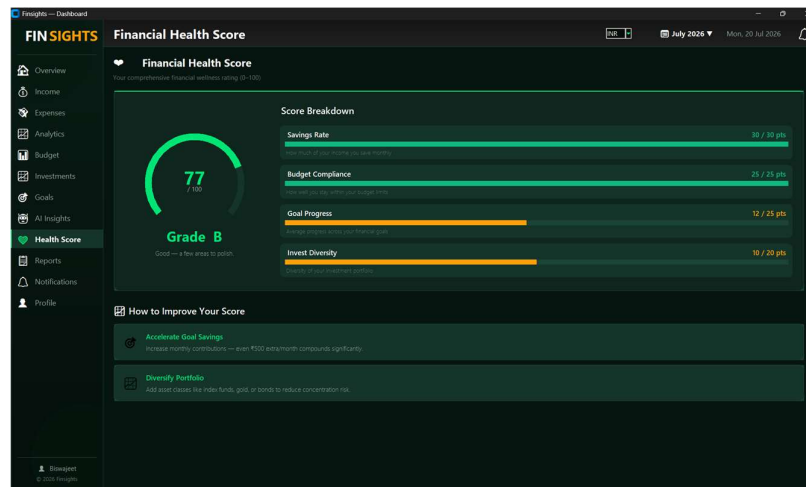


Fig 5.7 — Financial health score screen

5.5 Reports

The reports module lets users export a summary of their financial activity as a PDF (via ReportLab) or an Excel workbook (via OpenPyXL), with charts generated through Matplotlib and embedded into the export. This was one of the fiddlier parts of Milestone 2 — getting chart images to render correctly inside a generated PDF took a few passes, mainly around sizing and making sure the export path resolved correctly across different data states (e.g. a user with no transactions yet).

5.6 Notifications, Admin Panel and Profile

Notifications surface budget warnings and goal milestones inside the app itself. The admin panel is a separate view intended for oversight — reviewing registered users and sending broadcast messages — and is only reachable by an admin account. The profile section rounds out Milestone 2, letting a user edit their details, link additional accounts, change their password, and log out or delete their account.

6. Milestone 3: Intelligence & Insights

With the core record-keeping and planning features in place, Milestone 3 was about making the app actually tell the user something, instead of just storing what they typed in. This meant going back over the same transaction data used in Milestones 1 and 2 and building a layer on top of it that surfaces patterns — an analytics dashboard, a set of AI-style insights, and a more detailed version of the health score.

6.1 Analytics Dashboard

The analytics module pulls every transaction across the user's history and breaks it down into income vs. expense trends, a category-wise pie/bar view, and a savings-trend chart, all rendered with Matplotlib and embedded directly into the CustomTkinter window. I also added a compact summary table underneath the charts so a user can see the month-by-month numbers behind the visuals rather than only the picture. Getting this to look reasonable on both large and small screens meant writing a small formatting helper (`_short_num`) so that big totals collapse to a compact form like '1.2K' or '3.4M' instead of overflowing the card.

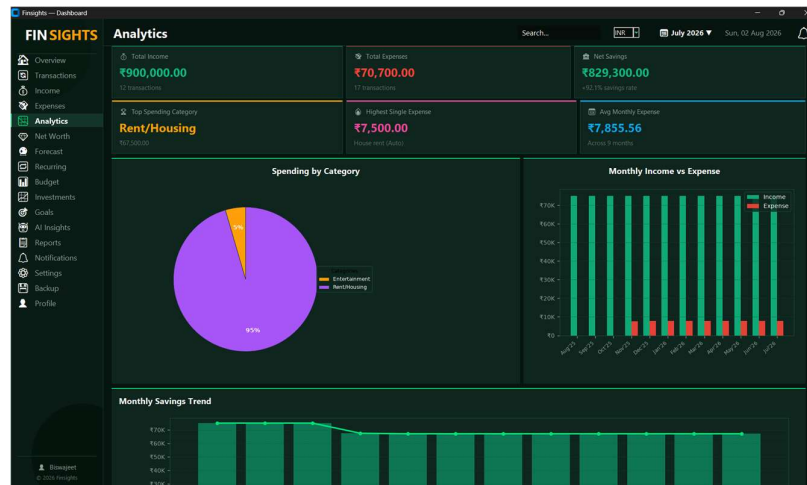


Fig 6.1 — Analytics dashboard

6.2 AI-Powered Insights

The insights screen looks at the same underlying data and generates a set of plain-language observations — things like which category is trending up, whether spending is outpacing income for the month, or which budget is closest to being blown. I call this 'AI-style' rather than claiming a full machine-learning model behind it, because at this stage it's a set of rule-based checks over the aggregated numbers rather than a trained model — but the presentation and the way it's surfaced to the user is deliberately styled the way a smarter recommendation engine would be, and it's a natural point to plug in something more sophisticated later.

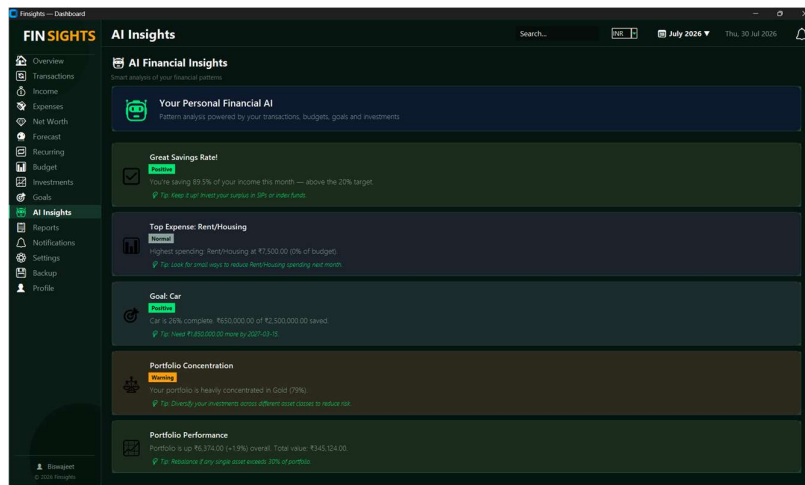


Fig 6.2 — AI-powered insights screen

6.3 Refined Financial Health Score

The health score from Milestone 2 got a rework here. Rather than a single number, the score is now paired with a letter grade (A through F) and a short description of what that grade means, plus a visual arc gauge instead of a plain progress bar. Under the hood the breakdown of contributing factors (spending discipline, savings rate, investment activity, debt/budget management) is shown separately, so the user can see exactly which factor is holding the overall number down instead of guessing.

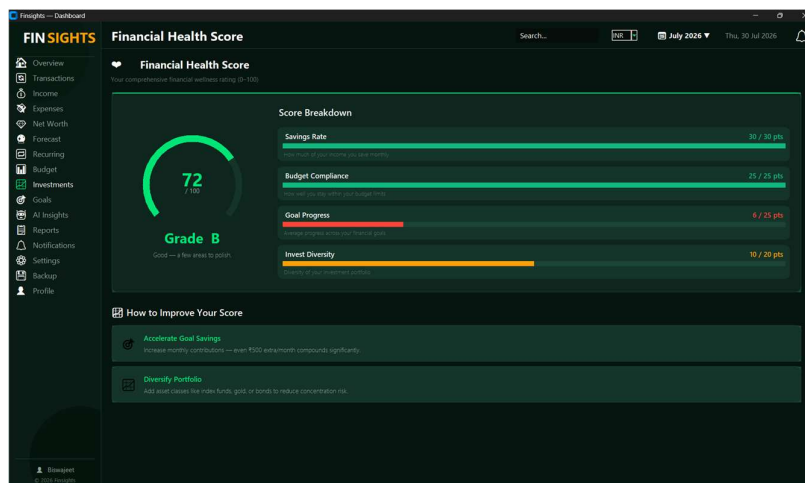


Fig 6.3 — Financial health score, overview

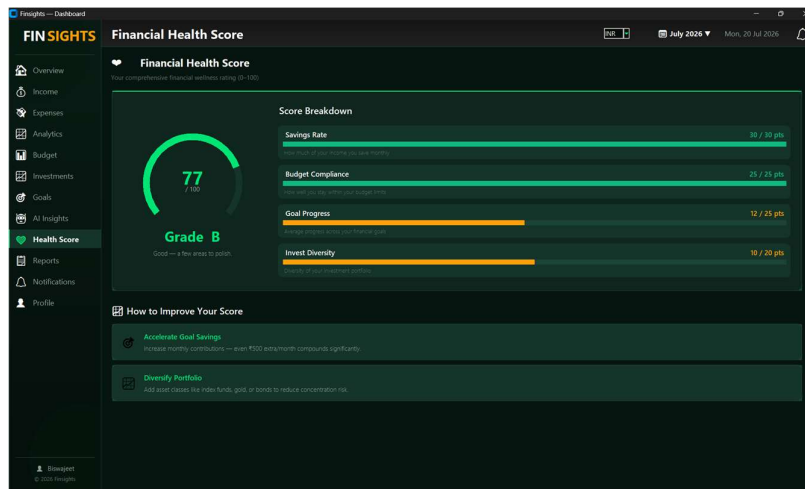


Fig 6.4 — Financial health score, factor breakdown

7. Milestone 4: Reporting, Automation & Finalization

Milestone 4 was the last stretch — rounding the app out with the features that make it something you'd actually keep using day to day rather than a one-time entry tool, plus general polish across the screens that had been sitting untouched since Milestone 1.

7.1 Advanced Reports

The reports module was extended further — the PDF and Excel exports now pull in the same charts used on the analytics screen, and the export flow was cleaned up so it handles edge cases like a user with no transactions yet, or a date range with no data, without crashing. I also added a proper reports overview screen so a user isn't just triggering a blind download but can see what's about to be exported first.

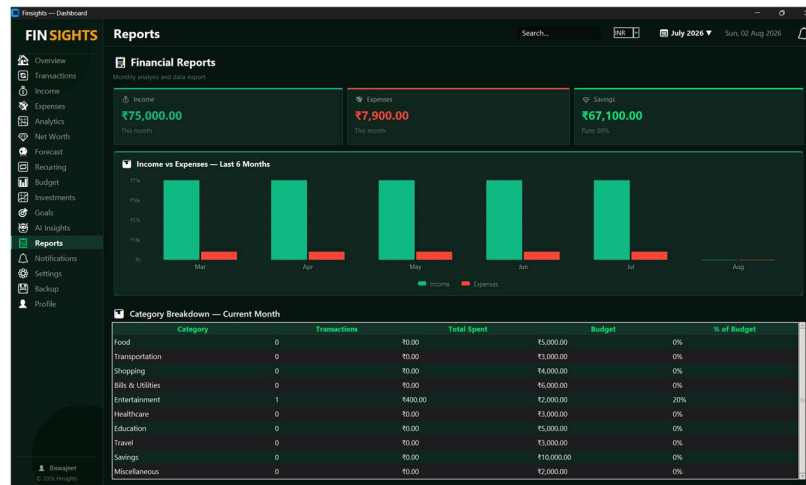


Fig 7.1 — Reports overview and export screen

7.2 Financial Forecast

The forecast module takes the monthly income and expense totals built up over time and projects them forward using a simple linear trend over the historical months. It deliberately refuses to guess when there isn't enough data — if a user has fewer than two months of transactions, it tells them so instead of showing a meaningless projection. This was one of the more interesting pieces to build precisely because of that constraint: the honest answer for a new user is 'not enough data yet', and the UI needed to handle that gracefully rather than just breaking.

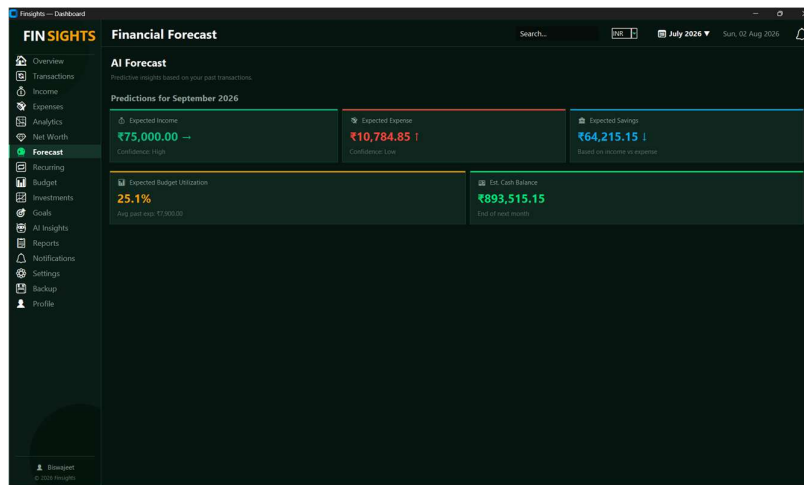


Fig 7.2 — Financial forecast screen

7.3 Net Worth Tracker

Net worth pulls together assets and liabilities that fall outside day-to-day transactions — things like property, savings accounts, or loans — and tracks the combined figure over time with a trend chart, similar in spirit to the investment tracker but working at the level of the user's overall financial position rather than individual holdings.

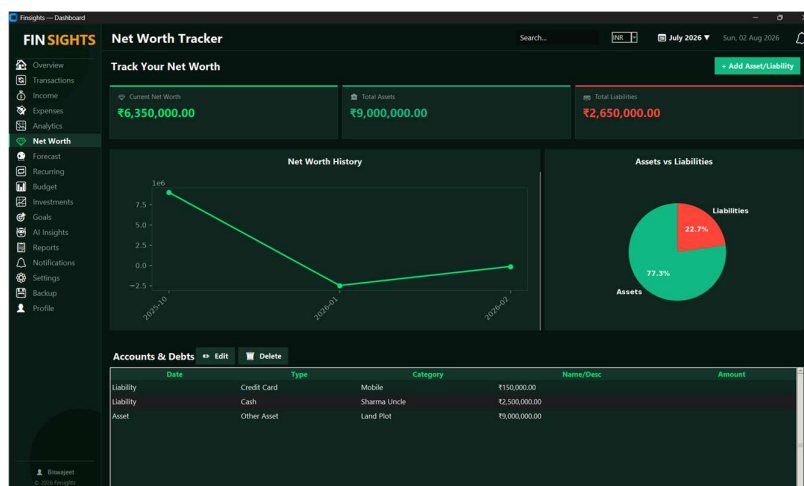
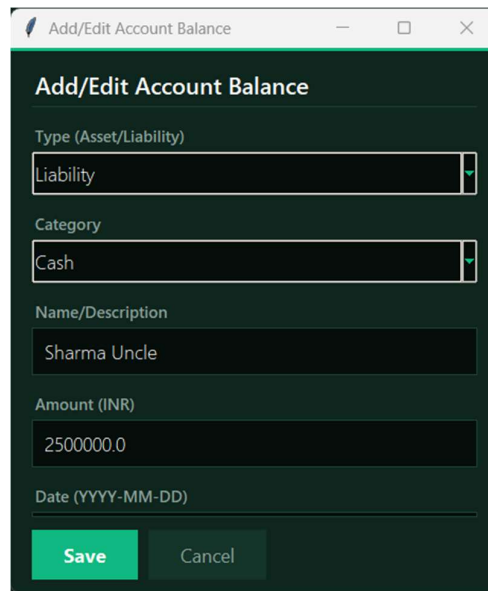


Fig 7.3 — Net worth tracker screen



Add/Edit Account Balance

Type (Asset/Liability)
 Liability

Category
 Cash

Name/Description
 Sharma Uncle

Amount (INR)
 2500000.0

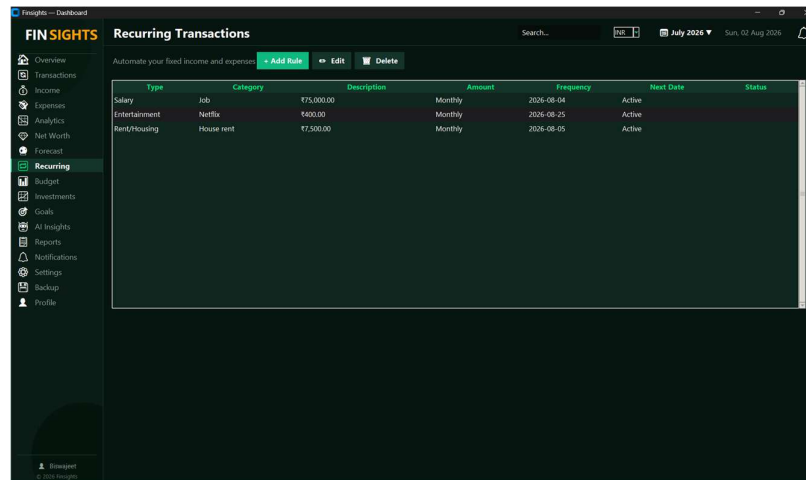
Date (YYYY-MM-DD)

Save **Cancel**

Fig 7.4 — Add net worth entry dialog

7.4 Recurring Transactions

A lot of real income and expense activity is repetitive — a monthly salary credit, a subscription, rent — so the recurring module lets a user define a rule once (amount, category, frequency) and have it generate the matching transaction automatically going forward, using python-dateutil to handle the date math across different frequencies. The generation logic runs a catch-up check on login, so if the app wasn't opened for a while it still backfills any recurring entries that were due in the meantime.



FIN SIGHTS **Recurring Transactions** Search... INR July 2026 Sun, 02 Aug 2026

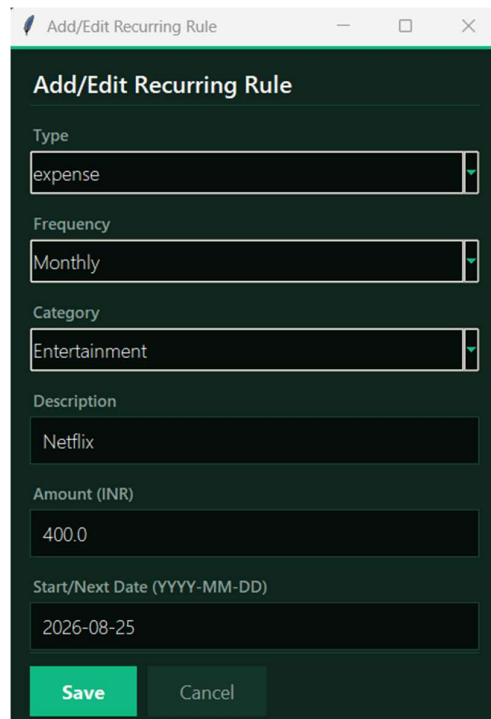
Automate your fixed income and expenses **+ Add Rule** **Edit** **Delete**

Type	Category	Description	Amount	Frequency	Next Date	Status
Salary	Job		₹75,000.00	Monthly	2026-08-04	Active
Entertainment	Netflix		₹400.00	Monthly	2026-08-25	Active
Rent/Housing	House rent		₹7,500.00	Monthly	2026-08-05	Active

Recurring **Budget** **Investments** **Goals** **AI Insights** **Reports** **Notifications** **Settings** **Backup** **Profile**

Emergency **2 min Insights**

Fig 7.5 — Recurring transactions screen



The dialog box is titled "Add/Edit Recurring Rule". It contains the following fields:

- Type:** A dropdown menu with "expense" selected.
- Frequency:** A dropdown menu with "Monthly" selected.
- Category:** A dropdown menu with "Entertainment" selected.
- Description:** A text input field containing "Netflix".
- Amount (INR):** A text input field containing "400.0".
- Start/Next Date (YYYY-MM-DD):** A text input field containing "2026-08-25".

At the bottom, there are two buttons: a green "Save" button and a grey "Cancel" button.

Fig 7.6 — Add recurring rule dialog

7.5 Backup & Restore

Since all data lives in local JSON files, losing the data folder means losing everything, so the backup module lets a user export the entire data directory into a single compressed ZIP file, and import it back in on the same or a different machine. The restore path deliberately warns the user before overwriting existing data, since that's a destructive action that's easy to trigger by accident.

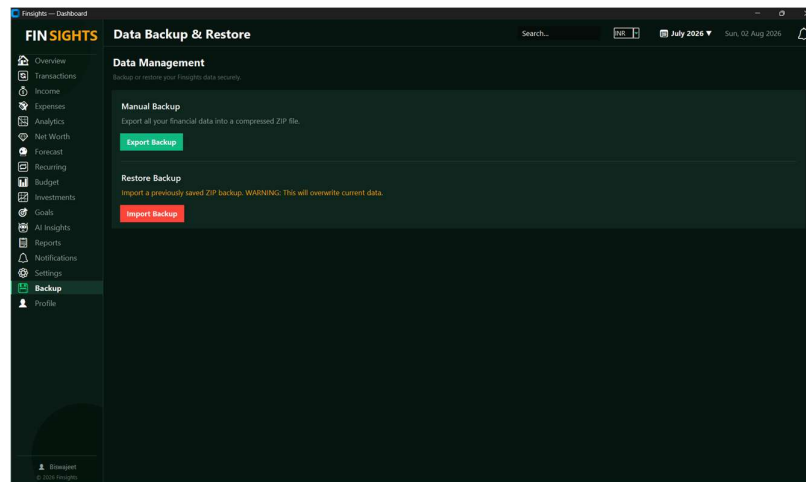


Fig 7.7 — Data backup and restore screen

7.6 Settings, Profile and Theming

The settings screen brings together preferences that were previously scattered or hardcoded — theme mode (dark, light or system), default currency, and whether budget alerts are shown — into one place. Adding a proper light theme in particular meant going back through the earlier modules and making

sure hardcoded colours were actually pulled from the theme configuration rather than fixed values, which was a good forcing function to clean up some early shortcuts from Milestone 1.

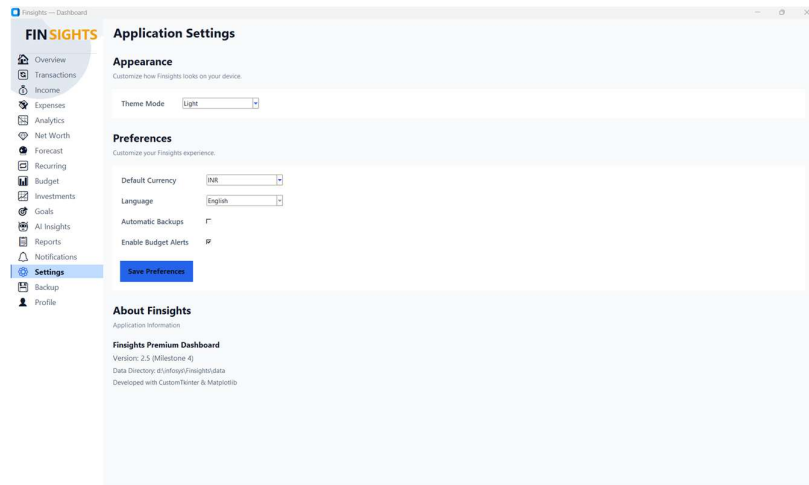


Fig 7.8 — Application settings screen

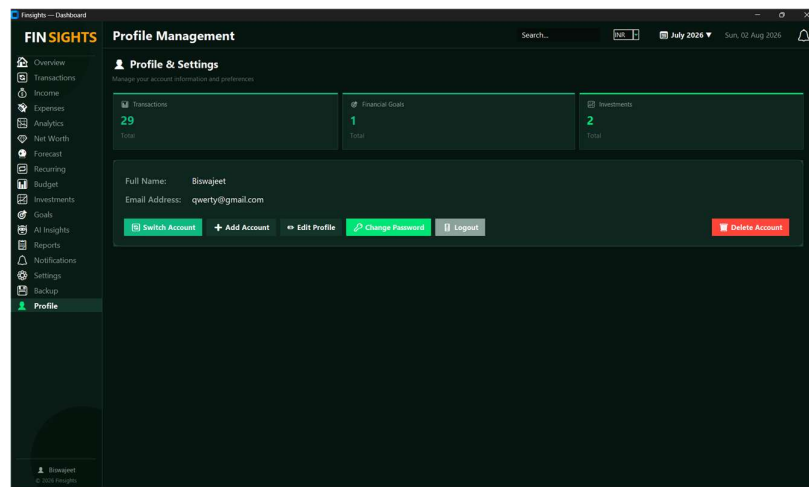


Fig 7.9 — Updated profile screen

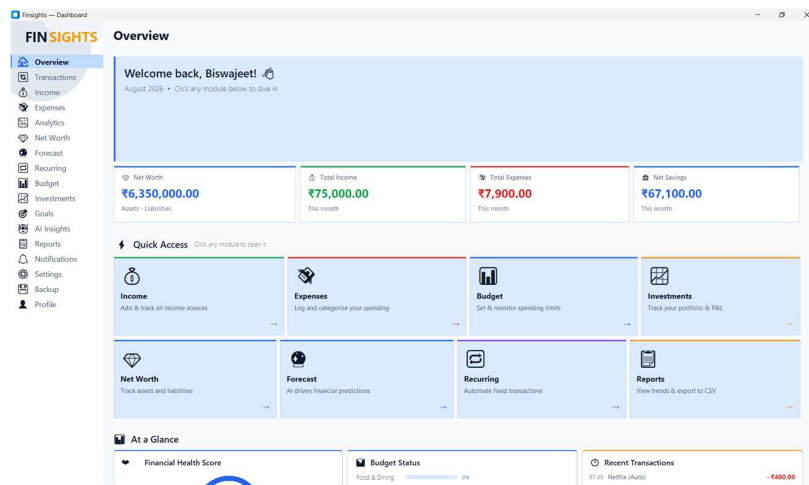


Fig 7.10 — Dashboard in light theme

8. Challenges Faced

- CustomTkinter doesn't expose every styling hook that plain Tkinter/ttk does, so a few widgets (like the Treeview tables) had to be styled through `ttk.Style()` separately, which meant keeping two theming systems visually consistent by hand.
- Getting the mouse-wheel scroll binding to work reliably across nested scrollable frames took a few iterations — an earlier version leaked bindings and caused the scroll to lock up after switching sections a few times.
- Deciding on a JSON-file storage approach instead of a proper database was a deliberate simplification for this stage, but it meant writing my own small set of load/save/id-generation helpers rather than relying on an ORM.
- Balancing the financial health score formula so it felt meaningful rather than arbitrary needed a couple of rounds of testing with sample data before the numbers looked reasonable.
- Embedding Matplotlib charts inside both the CustomTkinter dashboard and the exported PDF/Excel reports required two different rendering paths, which took some trial and error to keep visually consistent.
- Introducing a light theme in Milestone 4 surfaced a handful of hardcoded colours left over from earlier modules that only looked right against the dark background, so the theming pass took longer than expected.
- The recurring-transaction catch-up logic needed careful handling of the date-diff math (via `relativedelta`) to avoid double-generating entries if the app was opened more than once on the same day.

9. Conclusion and Future Scope

By the end of Milestone 4, FinSights covers the full loop of a personal finance workflow: registering an account, logging income and expenses, setting budgets, planning goals, tracking investments and net worth, automating recurring entries, seeing a consolidated and detailed health score, reading AI-style insights, forecasting ahead, exporting reports, and backing up all of it — all running locally without any external dependency. Building it module-by-module across four milestones made it easier to keep scope under control between sessions, which mattered given this was developed incrementally rather than in one continuous sprint.

There's still room to grow — a genuine migration from the current JSON files to the SQLite schema already designed on paper if the app starts to show its limits with larger datasets, optional cloud sync for backups, and a properly trained model behind the insights engine instead of the current rule-based checks are all reasonable next steps. For now, though, all four milestones together form a complete, working, self-contained personal finance application that does what it set out to do.

References

- CustomTkinter Documentation — <https://customtkinter.tomschimansky.com/>

- Python Software Foundation — Tkinter Documentation — <https://docs.python.org/3/library/tkinter.html>
- Matplotlib Documentation — <https://matplotlib.org/stable/>
- ReportLab User Guide — <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- OpenPyXL Documentation — <https://openpyxl.readthedocs.io/>